
OPTIMIZING PREFIX ATTENTION WITH THUNDERKITTENS KERNELS

Ayush Kumar¹ Jackson Romero¹ Jason Wei¹

ABSTRACT

Large language model inference workloads frequently consist of processing batches of sequences sharing a common prefix. Our project aims to implement an automatic prefix caching (APC) approach to KV cache reuse using highly performant kernels written in ThunderKittens (TK), a domain-specific language for GPU programming. We design and evaluate TK kernels built for existing approaches to prefix caching, as presented in the Hydragen and ChunkAttention frameworks. Our goal in this project is to try to use TK’s tile-level optimizations and explicit primitives for Tensor Core utilization and improve the throughput of existing prefix attention mechanisms. We evaluate candidate TK kernels for each approach, presenting results at different input batch/sequence lengths, prefix configurations, and an analysis of our performance against existing benchmarks. Our presentation is available [here](#).

1 INTRODUCTION

As large language models (LLMs) are utilized in a multitude of scenarios for multi-step and context-rich tasks such as code generation, there is a growing interest in optimizing inference at scale. Given that certain tasks require more complex reasoning (due to task difficulty) or a multi-step chain of thought (due to compound tasks), most providers are opting for inference-time augmentations such latent thinking steps (Wei et al., 2023) and few-shot prompting (Brown et al., 2020).

These inference-time steps, while improving model performance, introduce a critical computational bottleneck: many sequences processed in a single batch share substantial common prefixes. For instance, in few-shot prompting scenarios, dozens of user queries may be prepended with identical demonstration examples. Similarly, agentic workflows and structured generation tasks often involve repeated system prompts or contextual information across multiple requests. The redundant computation of attention over these shared prefixes represents a significant inefficiency in modern LLM serving systems.

Modern implementations of the transformer architecture utilize a key-value cache (Pope et al., 2022) to store attention keys and values to avoid recomputation across autoregressive generation steps. By recognizing and exploiting prefix overlap across sequences in a batch, inference systems can dramatically reduce both computational costs and memory bandwidth requirements.

ChunkAttention (Ye et al., 2024) proposed prefix-aware KV cache and an efficient 2-phase partition approach to computing self-attention. Similarly, Hydragen (Juravsky et al.,

2024) proposes a decomposition and sequence batching approach to compute attention optimally across the suffix and the prefix.

While there are other approaches to managing shared prefixes, we choose to work with the above two frameworks because their attention calculation consists of a prefix-suffix step, which can be represented (non-optimally) as multiple matrix-vector products. This computation does not use the tensor cores available in modern GPUs such as the H100 efficiently. While both ChunkAttention and Hydragen do batching of the shared prefix queries, tensor core utilization can be improved by modifying their attention kernels to make use of better tile-level abstractions that better map tensor operations at the hardware level.

We opt to rewrite the kernels being used for attention in these frameworks with Thunder Kittens (TK) (Spector et al., 2024). TK’s various tile-level optimizations and explicit primitives for Tensor Core utilization allow us to improve hardware utilization and reduce latency during the decoding phase.

2 PROBLEM

The core problem that our project aims to solve is the underutilization of tensor cores in prefix-aware attention computation. There are several bottlenecks in the current implementations of ChunkAttention and Hydragen, which we hope to improve.

2.1 Redundant Computation in Prefix-Aware Decoding

Standard attention computes independently for each sequence, leading to redundant computation and memory access patterns. Hydragen and ChunkAttention enable efficient attention computation across a shared prefix. This helps speed up the prefill phase, where the prompt prefix is often shared across multiple independent queries. The problem, however, stems from the decoding phase, where the system computes attention over mismatched suffix tokens. Current approaches use regular self-attention kernels for this phase, which do not always provide good utilization of Tensor Cores. We propose to solve this problem by writing specialized kernels for prefix-aware decoding using TK.

2.2 Hardware Underutilization

Modern GPUs such as the H100 feature specialized tensor cores capable of significantly improving throughput for matrix multiplications than traditional CUDA cores. However, current prefix-aware attention implementations such as in ChunkAttention and Hydragen rely on general-purpose kernels (e.g. FlashAttention). This might be because the implementations target broader hardware compatibility. We aim to improve utilization on specialized hardware such as the H100 using custom TK kernels.

2.3 Implementation Complexity

Writing specialized kernels for Tensor Core utilization introduces significant implementation complexity if there is not much support at the kernel language level. We choose TK because it provides an extensible framework on top of CUDA to operate on tiles with useful hardware primitives, such as asynchronous warp-level matrix multiply-and-accumulate (WGMMMA) calls, and register/shared tiles. We think that TK strikes a good abstraction balance in that we can access complex hardware instructions but fall back to the tile-based abstraction as our mental model. This allows us to try to write the attention kernels in a more TMA-optimal manner without having to rely on complex hardware instructions.

3 RELATED WORK

Existing work in the area of automatic prefix caching (APC) and KV cache reuse include PagedAttention (Kwon et al., 2023), ChunkAttention, and Hydragen. PagedAttention borrows from ideas in virtual memory to avoid redundant storage of prefixes, using page-level storage of KV tensors. However, PagedAttention does not prevent redundant reads of the prefix’s keys and values from GPU memory. ChunkAttention uses a prefix-aware KV cache and the two-phase partition kernel for computing self-attention,

which splits the attention computation into the shared prefix (chunk-first phase) and the unique suffix (sequence-first). The limitation of this approach is that the sequence-first approach attends to unique suffix tokens as query vectors, instead of matrices, which is critical as it does not allow us to exploit full utilization of the matmul Tensor Cores. Also, the kernel used in this approach is a highly specialized CUDA kernel tuned to a specific hardware and model configuration. Using a TK kernel would allow us to port to any hardware with Tensor Cores and would also allow developers to code in a native PyTorch-like environment. Finally, Hydragen utilizes an attention decomposition and sequence batching approach to manage attention between suffix and prefix. Hydragen uses a Triton kernel for this purpose. We anticipate that using TK’s various tile-level optimizations and explicit primitives for Tensor Core utilization will allow us to further reduce latency during the decoding phase.

3.1 Hydragen

Hydragen targets the common LLM serving regime where many sequences share a long prefix but diverge in their suffix. It decomposes the attention computation into a shared-prefix component and a unique-suffix component, and then combines the resulting partial outputs using log-sum-exp (LSE) rescaling. A key systems insight is that shared prefix attention can be evaluated with inter-sequence batching to convert many memory-bound matrix-vector products into fewer compute-efficient matrix-matrix multiplications. Hydragen’s reference implementation relies on FlashAttention-style kernels for the underlying attention primitives; our work explores whether ThunderKittens’ warpgroup tensor core primitives can improve the performance of these attention calls on H100-class GPUs, and where integration overhead or kernel constraints limit end-to-end speedups.

3.2 ChunkAttention

ChunkAttention addresses prefix-aware attention through a two-phase partition approach that separates computation across shared prefixes and unique suffixes. The algorithm divides the attention computation into two distinct phases:

Phase 1: Chunk-First. In this phase, the system processes shared prefix chunks by batching queries from multiple sequences together. Given a batch of sequences sharing a common prefix, the prefix is divided into fixed-size chunks (typically 64 tokens). For each chunk, queries from all sequences are processed together as a matrix operation against the chunk’s keys and values. This transforms what would otherwise be n independent matrix-vector products into a single matrix-matrix multiplication, enabling efficient utilization of GPU tensor cores. The phase outputs partial attention results (unnormalized attention outputs, max values, and sum values) for each sequence-chunk pair.

Phase 2: Sequence-First. This phase handles the unique suffix tokens where sequences diverge. Each sequence is processed independently, computing attention over its suffix tokens one sequence at a time. The kernel reuses the cached partial results from the chunk-first phase for the shared prefix and computes fresh attention for the unique suffix tokens. Online softmax rescaling combines the prefix and suffix attention outputs to produce the final result.

The two-phase decomposition exploits the key insight that shared prefixes enable batched computation (compute-bound), while unique suffixes require per-sequence processing (memory-bound). ChunkAttention-TK implements both phases using ThunderKittens kernels to maximize hardware utilization on modern GPUs.

4 OVERVIEW

As our project aims to improve existing implementations for shared prefix attention at the kernel-level, we do not propose any modifications to the core algorithms proposed by the Chunk Attention and Hydragen frameworks.

At a high level, our project consists of two parts, Hydragen-TK, our implementation of a TK kernel for Hydragen, and ChunkAttention-TK, a TK kernel for ChunkAttention.

4.1 System Overview: Hydragen-TK

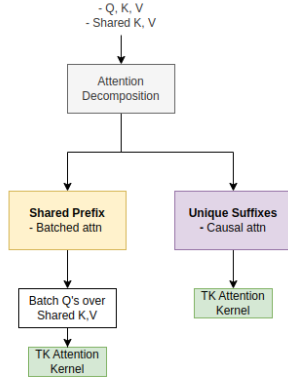


Figure 1. Hydragen TK Overview

The Hydragen-TK framework is summarized in the above overview. We continue to use the attention decomposition step presented in the original paper, and use TK attention kernels in both computing attention over the shared prefix as well as the unique suffixes.

4.2 System Overview: ChunkAttention-TK

4.2.1 Overview: ChunkAttention Algorithm

The ChunkAttention algorithm partitions attention computation across shared and unique portions of input sequences. As described in Section 3.2, the chunk-first phase processes shared prefix chunks using batched matrix operations, while the sequence-first phase processes unique suffix tokens using per-sequence computation with cached prefix results.

The original ChunkAttention implementation uses hand-optimized CUDA kernels with Warp Matrix Multiply Accumulate (WMMA) instructions, fp16 precision, and 128 threads (4 warps) per block. Our contribution reimplements both kernels using ThunderKittens to explore whether TK’s tile-based abstractions and warpgroup primitives can improve performance while simplifying the implementation.

4.2.2 ThunderKittens Kernel Implementation

We developed two TK kernels corresponding to the two-phase partition:

Chunk-First Kernel (`attn_chunk_first_tk`): This kernel computes attention for a single chunk of the shared prefix across all sequences in the batch. The computation follows the standard attention pattern: $\text{scores} = QK^T \cdot \text{scale}$, followed by row-wise softmax (outputting max and sum for later rescaling), and finally $\text{attn} = \text{softmax}(QK^T)V$. The kernel leverages TK’s warpgroup matrix multiply (`warpgroup::mm_ABt` and `warpgroup::mma_AB`) for the two matrix multiplications and warp-level primitives for softmax operations. The grid is configured as (n_heads, n_chunks) with 4 warps per block.

Sequence-First Kernel (`attn_seq_first_tk`): This kernel computes the final attention output for a single sequence by combining cached chunk-first results with fresh computation over unique suffix tokens. The kernel processes chunks in a round-robin fashion across warps: for shared chunks, it retrieves cached attention outputs and uses online softmax rescaling to merge them; for unique chunks, it computes fresh attention using scalar dot products for QK^T (since each query is a single vector). The grid is configured as (n_heads, n_seqs) with 4 warps per block.

Both kernels use bf16 precision for improved H100 compatibility and TK’s shared memory allocators for proper tile layout and swizzling.

5 METHODOLOGY

5.1 Hydragen-TK

5.1.1 Overview: Hydragen Algorithm

As mentioned above, Hydragen optimizes LLM inference for batches with shared prefixes using two key techniques:

1. **Attention Decomposition:** The attention computation is split into separate computations over shared prefixes and unique suffixes, which are combined using log-sum-exp values.
2. **Inter-Sequence Batching:** For shared prefix attention, queries from different sequences are batched together. This transforms memory-bound matrix-vector products into compute-bound matrix-matrix products.

The original Hydragen implementation uses PyTorch for decomposition logic and Triton for attention kernels. Our contribution is re-implementing the attention kernels using ThunderKittens to explore whether TK’s tile-based abstraction and explicit Tensor Core primitives can provide comparable performance with simpler code.

The log-sum-exp is defined as

$$\text{LSE}(Q, K) = \log \sum_{j=1}^{N_k} \exp \left(\frac{(QK^T)_{ij}}{\sqrt{d}} \right) \quad (1)$$

As is presented in the paper, we can take the attention outputs and the LSEs from the partitioned computation and using rescaling, compute the the full-sequence softmax denominator.

5.1.2 ThunderKittens Kernel Implementation

We adopt the use of the ThunderKittens’ attention kernel `tk.mha_forward`. TK imposes hardware-specific constraints for its H-100 optimized multi-head attention kernel that we discovered through experimentation and limited documentation such as:

- Self-attention requiring matching query-key sequence lengths
- 192-token alignment (tile size) for the attention kernel
- `bfloat16` data type for all inputs
- 64/128-dim attention heads required by TK’s API for `tk.mha_forward`

We modified the Hydragen source to launch the MHA kernel defined in TK instead of the Triton FlashAttention kernel as is in the original code. We utilized a padding to satisfy the

192-token alignment as required by the TK kernel, and setup additional checks to make sure that the instance satisfies the other runtime constraints before launching the TK kernel. As a fallback, we call the Triton Flash Attention kernel as in the original implementation.

In practice, we also observed additional stability limitations on certain larger sequence lengths. To keep evaluation reproducible, we report both end-to-end results (where the system falls back to FlashAttention when TK cannot be used) and separate kernel-only microbenchmarks for configurations where TK is stable.

As TK’s documentation is currently still a work in progress, we were limited to a few resources such as this [onboarding document](#). It was not a trivial task to get specific documentation for parts of the API, and we used the onboarding document as well as the sample kernels provided in the TK public GitHub as a reference when implementing our kernels.

5.1.3 TK Adapter for Hydragen

In order to integrate the TK kernel correctly into Hydragen’s interface, we needed to build a minimal adapter layer to act as a bridge between the actual and expected inputs. The steps involved in the transformation of the inputs and outputs are as follows:

1. Cast inputs to `bfloat16`
2. Transpose inputs $(B, N, H, D) \rightarrow (B, H, N, D)$
3. Do a similar (reverse) transpose in the outputs
4. Process the LSE in the output by squeezing the last dimension and transposing LSE $(B, H, N, 1) \rightarrow (B, N, H)$

An additional optimization we tried at input preparation stage was using `.contiguous()` to ensure contiguous memory for the queries, keys, and values to maximize data locality. With this optimization, we were able to get minimal (<5% speedup), implying that data locality was not the bottleneck in the implementation.

5.2 ChunkAttention-TK

5.2.1 Chunk-First Kernel Implementation

The chunk-first kernel processes a single chunk of the shared prefix across all sequences in a batch. We configure the kernel with a grid of (n_heads, n_chunks) and 4 warps per block (128 threads), matching the original implementation.

Memory Management and Data Loading. A key challenge arose from ChunkAttention’s KV cache organization: keys and values are stored as arrays of void pointers

(void**) to support flexible per-chunk allocation. TK’s Tensor Memory Accelerator (TMA) requires contiguous global memory with compile-time strides, making TMA incompatible with this pointer-based layout. We addressed this by implementing a hybrid loading strategy: Queries use TMA (via `tma::load_async`) since they have regular strided layout, while Keys and Values use custom asynchronous copy routines with `cp.async` instructions.

To reduce shared memory consumption, we reuse the same buffer for K and V tiles—they are never needed simultaneously since K is consumed during QK^T computation before V is loaded for the final matrix multiply. This optimization reduces shared memory usage by 40% compared to separate allocations.

Warpgroup Matrix Operations. The kernel leverages TK’s warpgroup-level matrix multiply primitives for the two main computations. For QK^T , we use `warpgroup::mmABt(scores_r, Q_s, KV_s)` which maps directly to H100’s WGMMMA instructions. After computing softmax using warp-level primitives (`warp::row_max`, `warp::exp`, `warp::row_sum`), we compute the final attention output with `warpgroup::mma_AB(out_r, exp_scores, KV_s)`.

This design contrasts with the native CUDA kernel, which uses hand-written WMMA fragment operations and manual shared memory management with custom padding. TK’s tile abstractions automatically handle swizzling for bank conflict avoidance and provide higher-level warpgroup operations, simplifying the implementation while enabling better instruction selection on H100.

5.2.2 Sequence-First Kernel Implementation

The sequence-first kernel computes the final attention output for a single sequence by combining cached chunk-first results with fresh computation over unique suffix tokens. We configure the grid as (n_heads, n_seqs) with 4 warps per block.

Scalar Dot Product Pattern. Each sequence processes one query vector against multiple chunks, resulting in a fundamentally different computational pattern than chunk-first. For each chunk, the kernel computes QK^T using scalar dot products: we iterate over each row of K and perform element-wise multiplication with Q, followed by warp-level reduction (`_shfl_down_sync`) to sum the products. This row-by-row processing cannot leverage matrix-tiled operations or tensor cores.

Online Softmax and Result Merging. Warps process chunks in round-robin fashion. For shared prefix chunks (`chunk_idx < n_shared_chunks`), the kernel retrieves cached partial results (max, sum, unnormalized attention)

from chunk-first and uses online softmax rescaling to merge them:

$$\begin{aligned} m_{\text{new}} &= \max(m_{\text{prev}}, m_{\text{cached}}) \\ \text{scale}_{\text{prev}} &= \exp(m_{\text{prev}} - m_{\text{new}}) \\ \text{scale}_{\text{cached}} &= \exp(m_{\text{cached}} - m_{\text{new}}) \\ \text{out} &= \text{out} \cdot \text{scale}_{\text{prev}} + \text{cached_attn} \cdot \text{scale}_{\text{cached}} \end{aligned}$$

For unique suffix chunks, the kernel computes fresh attention using the scalar dot products described above. TK’s warp primitives (`warp::max`, `warp::sum`, `warp::exp`) simplify this online softmax implementation compared to the native kernel’s manual shuffle operations, though they cannot overcome the fundamental limitation: without matrix-matrix multiplies, tensor core utilization remains minimal.

After all chunks are processed, warp 0 performs cross-warp reduction to normalize the final output and writes the result to global memory.

5.2.3 Correctness Validation

We validated both kernels against the native CUDA implementations using 36 test configurations spanning different batch sizes (16-128), chunk counts (4-64), and head dimensions (128). We consistently observed some tests failing, but believe that this is due to numerical precision. All of our tests that use small-valued tensors pass and only when using values across the whole range of `bf16` did we indices with noticeable, but still small, differences. The test harness runs a basic PyTorch-based computation to check equivalence.

6 EVALUATION

6.1 Hydragen-TK

6.1.1 Correctness

Given that our project attempts to swap out existing kernels for optimized TK kernels in the Hydragen framework, along with any relevant speedup, we needed to verify the correctness of our implementation. We implemented two test scenarios in to verify correctness:

1. **Unique Prefill (Self-Attention):** This test validates that the kernel produces numerically correct results for standard self-attention during the prefill phase, without using shared prefixes. The purpose of this test was to make sure that the invocation of Tk self-attention kernels was correct using the appropriate tensor shapes and parameters.
2. **Shared Prefix Attention:** In this test, we used standard long prompt where Q and K are the same length and checked for numerical equivalence between the

reference implementation (provided in the Hydragen code) as well as our implementation of shared prefix attention. This test ensures the correctness of our attention kernel when used after attention decomposition in Hydragen-TK.

In both implementations, we tested with references using PyTorch and Triton attention kernels and used a tolerance for error of 0.05.

6.1.2 Experimental Setup

- Hardware: H100 GPU
- Framework: PyTorch 2.8, ThunderKittens version 3.0
- Baseline: FlashAttention-2
- Configuration: B=16, S=768, H=16, D=128, TK using bfloat16

6.1.3 Performance Results

Table 1. Latency Comparison (100 iterations, 10 warmup)

Implementation	Latency (ms)
Hydragen-TK (H100)	0.85 ± 0.01
FlashAttention (Baseline)	0.59 ± 0.01

Table 2. Throughput Comparison (100 iterations, 10 warmup)

Implementation	Throughput (TFLOPS)
Hydragen-TK (H100)	91.09
FlashAttention (Baseline)	130.62

From the above results, we can see that the modified TK kernels did not provide any speedup: rather, they incurred 40% overhead when compared to FlashAttention-2. The reason for this could be the expensive input/output data transformations in the adapter required to bridge between Hydragen and TK’s `tk.mha_forward` kernel. Since the extra transformation steps include several reshapes, as well as a cast, this results in a higher overhead in launching each kernel.

In order to tune the performance of our TK kernel, we tried several approaches. First, we enforced that the key, values and queries were stored in contiguous storage, and optimization that provided upto 5% speedup.

6.1.4 Kernel-Only Benchmarks (Native vs TK)

To isolate kernel behavior from Python-side adapter and framework overheads, we additionally benchmark the underlying attention kernels using CUDA-event timing (kernel-only). Figure 2 compares the native attention kernel

(FlashAttention call) against the TK attention kernel on an H100, for a TK-stable configuration. We find that, when TK can be invoked successfully, the TK kernel can outperform the native kernel on this workload.

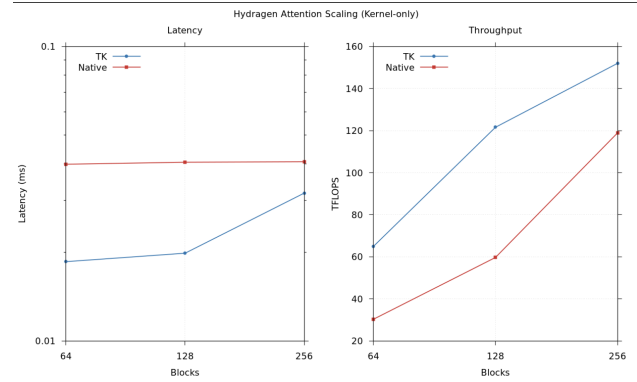


Figure 2. Hydragen kernel-only comparison (Native vs TK). We report latency and throughput scaling versus a simple grid proxy (blocks). These results isolate the attention kernel from end-to-end Hydragen overheads.

6.1.5 Shared-Prefix Microbenchmark (Hydragen-Style)

Hydragen’s core optimization targets batches with shared prefixes. To better reflect this setting, we introduce a shared-prefix microbenchmark that measures attention against a shared KV prefix with inter-sequence batching. We report kernel-only measurements for native and TK variants in Figure 3. This benchmark complements the end-to-end numbers above by answering a narrower question: whether the TK kernel provides a faster primitive when the shared-prefix computation can be expressed in a TK-compatible shape.

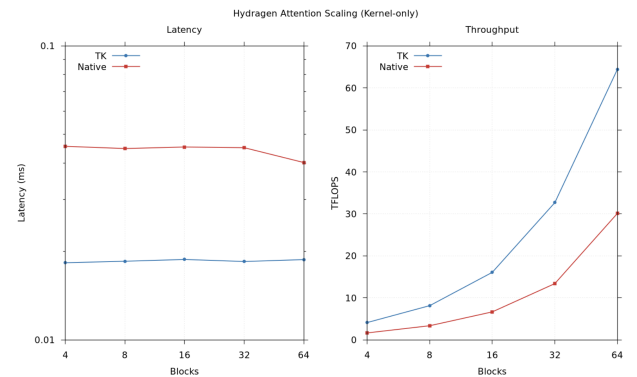


Figure 3. Hydragen shared-prefix kernel-only microbenchmark (Native vs TK). This benchmark reflects Hydragen’s shared-prefix setting and isolates kernel performance from adapter and orchestration overhead.

6.1.6 Discussion and Limitations

Taken together, the end-to-end results (Tables 1–2) and the kernel-only results (Figures 2–3) suggest that TK attention kernel can be competitive or faster than the native kernel when it is applicable, but end-to-end Hydragen performance can still degrade due to adapter overheads and limited kernel applicability. In particular, on our H100 testbed we observed that TK stability and supported shapes were more restrictive than the simple alignment and head-dimension constraints, which makes it difficult to claim uniform wins at larger sequence lengths without additional kernel work. Reducing the adapter cost (casts/transposes) and expanding stable shape coverage are promising directions for future improvements.

6.2 ChunkAttention-TK

6.2.1 Experimental Setup

- Hardware: H100 GPU (80GB HBM3)
- Framework: ThunderKittens 3.0, CUDA 12.4
- Baseline: Native CUDA kernels (fp16)
- Config: `n_seqs=64`, `chunk_size=64`, `d_head=128`, `bf16`
- Sweep: `n_heads` $\in \{4, 8, 16, 32, 64, 128, 256\}$, `n_chunks` $\in \{4, 8, 16, 32, 64, 128, 256\}$

We benchmark both the chunk-first and sequence-first kernels using kernel-only timing (CUDA events) to isolate kernel performance from framework overhead.

6.2.2 Chunk-First Kernel Performance

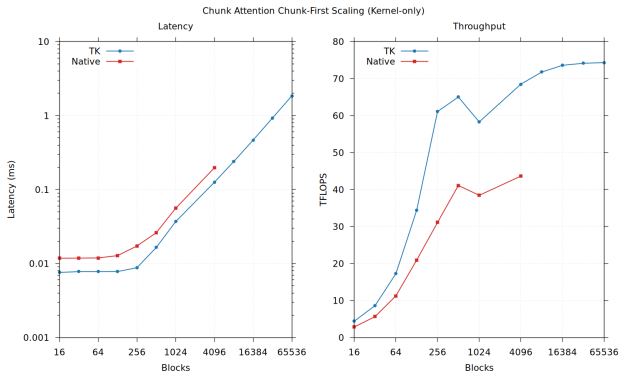


Figure 4. Chunk-First: TK vs Native kernel-only comparison. Left: Latency (ms) vs blocks. Right: Throughput (TFLOPS) vs blocks.

The TK chunk-first kernel demonstrates significant performance advantages over the native implementation across all scales (Figure 4). At 4,096 blocks, the TK kernel achieves

68.5 TFLOPS compared to 43.6 TFLOPS for native—a **1.6 $\times$ speedup**. The speedup is even more pronounced at smaller scales, reaching **2.0 $\times$** at 256 blocks.

This performance advantage indicates that TK’s warpgroup MMA operations provide better instruction throughput than the native kernel’s WMMA-based approach. The TK kernel achieves 7.5% of H100’s theoretical dense bf16 tensor core peak (989 TFLOPS), with throughput scaling nearly linearly as the workload grows. Memory bandwidth efficiency reaches 34.7% of peak HBM3 bandwidth at the largest scale, suggesting the kernel is transitioning from compute-bound to memory-bound behavior.

6.2.3 Sequence-First Kernel Performance

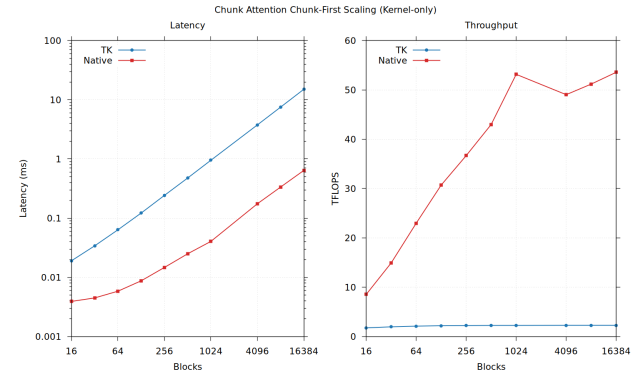


Figure 5. Sequence-First: Left: Latency (ms) vs blocks. Right: Throughput (TFLOPS) vs blocks.

The sequence-first kernel achieves significantly lower throughput than chunk-first, plateauing at approximately 2.3 TFLOPS (0.23% of peak) regardless of scale (Figure 5). This is an unfortunate consequence of how we designed the TK kernel: the kernel processes one query vector per sequence, resulting in vector-matrix products rather than matrix-matrix multiplies. Without matrix-tiled operations, the kernel cannot effectively utilize tensor cores.

Compared to chunk-first, sequence-first is approximately **30 $\times$ slower** in terms of TFLOPS (68.5 vs 2.3 at 4096 blocks). The sequence-first kernel’s performance is primarily memory-bandwidth-limited, as evidenced by its consistent throughput across scales. The scalar dot product pattern results in strided memory accesses that limit bandwidth utilization. While TK’s warp primitives simplify the implementation compared to the native kernel’s manual shuffle-based reductions, they cannot overcome the fundamental compute pattern mismatch with tensor core architecture.

TK will swizzle memory layouts by default as it expects most data to be loaded in chunks for eventual matmuls. This kernel breaks both of those design assumptions, and as such

we get all of the overhead with none of the benefit.

6.2.4 Overall Assessment

The chunk-first TK kernel successfully leverages H100’s warpgroup MMA capabilities to outperform the native CUDA implementation by up to 2x, demonstrating that TK’s tile abstractions can enable both simpler code and better performance for matrix-heavy kernels. The sequence-first kernel, while correct and functionally complete, illustrates the challenges of applying TK to memory-bound, non-tiled workloads where high-level abstractions provide less benefit.

7 CONCLUSION

In this project, we explored whether ThunderKittens’ tile abstractions and warpgroup tensor core primitives can improve performance for prefix-aware attention workloads on H100 GPUs. We implemented and evaluated TK-based kernels for two complementary prefix caching frameworks: ChunkAttention and Hydragen.

For ChunkAttention, we found that the chunk-first phase—whose computation naturally maps to matrix-matrix operations—benefits substantially from TK’s warpgroup MMA primitives, yielding up to $2\times$ kernel-only speedups over the native CUDA baseline in our sweeps. In contrast, the sequence-first phase is fundamentally dominated by vector-matrix dot-product patterns and irregular memory access, limiting tensor core utilization and diminishing the benefit of TK abstractions.

For Hydragen, our results highlight a key systems trade-off: while TK can provide a strong attention primitive in controlled kernel-only settings (including a shared-prefix microbenchmark), end-to-end performance can be bottlenecked by integration overheads such as dtype/layout conversions and by restrictive kernel applicability. These findings suggest that extracting consistent system-level wins requires co-design across kernel constraints, cache layout, and the integration boundary.

Future work includes expanding stable TK shape coverage for longer sequence lengths, reducing adapter overhead via layout-aware KV caching or fused transforms, and integrating a fully end-to-end Hydragen evaluation (shared-prefix + unique-suffix) across a broader sweep of serving-relevant workloads.

REFERENCES

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu,

J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners, 2020. URL <https://arxiv.org/abs/2005.14165>.

Juravsky, J., Brown, B., Ehrlich, R., Fu, D. Y., Ré, C., and Mirhoseini, A. Hydragen: High-throughput llm inference with shared prefixes, 2024. URL <https://arxiv.org/abs/2402.05099>.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.

Langley, P. Crafting papers on machine learning. In Langley, P. (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference, 2022. URL <https://arxiv.org/abs/2211.05102>.

Spector, B. F., Arora, S., Singhal, A., Fu, D. Y., and Ré, C. Thunderkittens: Simple, fast, and adorable ai kernels, 2024. URL <https://arxiv.org/abs/2410.20399>.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL <https://arxiv.org/abs/2201.11903>.

Ye, L., Tao, Z., Huang, Y., and Li, Y. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition, 2024. URL <https://arxiv.org/abs/2402.15220>.

Put anything that you might normally include after the references as an appendix here, *not in a separate supplementary file*. Upload your final camera-ready as a single pdf, including all appendices.